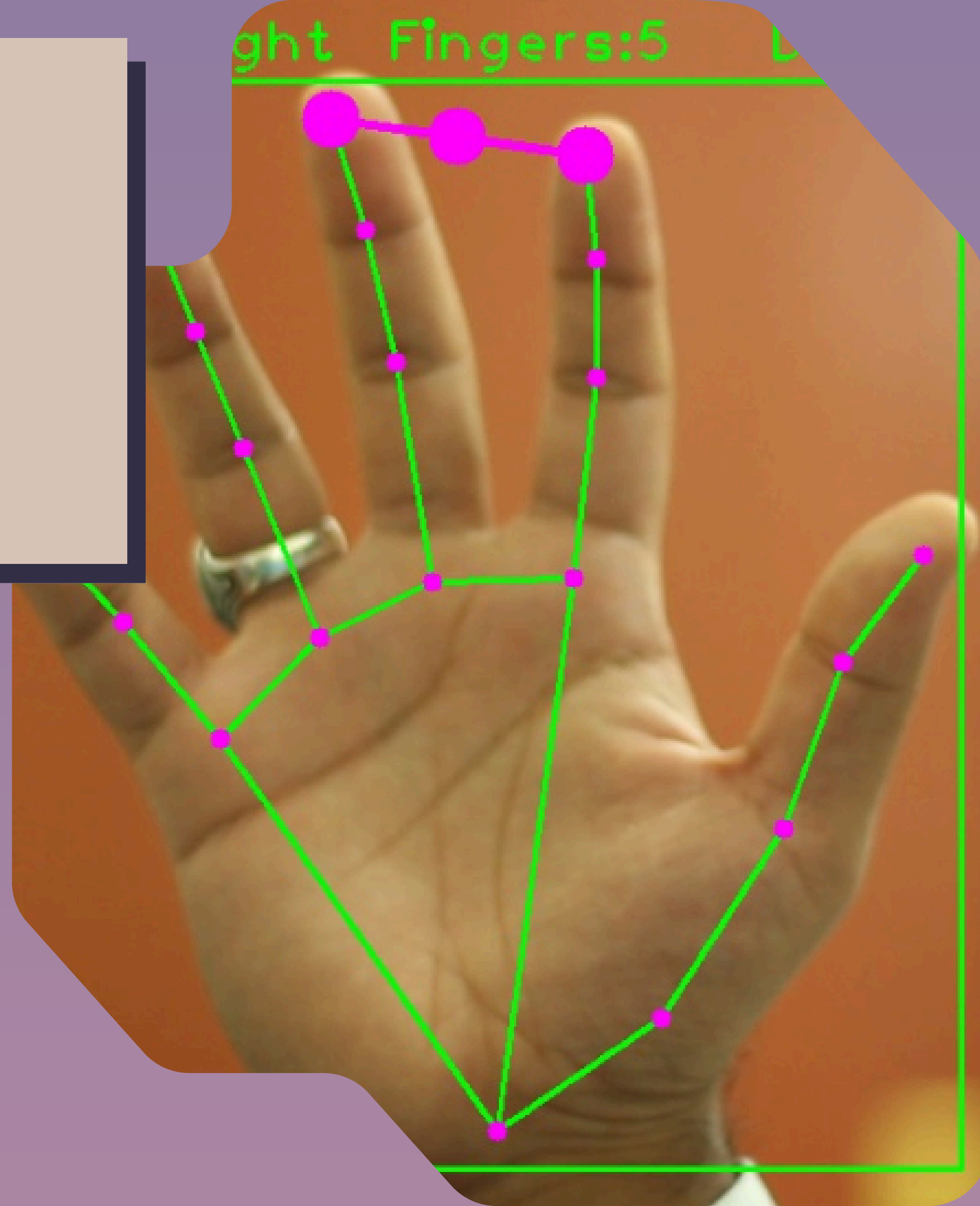


Project

Hand Gesture Controlled Paint Application - Project Report



1. Problem Statement

Traditional drawing applications require physical input devices like mice, styluses, or touchscreens, creating barriers for users with motor impairments, those in sterile environments, or scenarios needing hands-free operation. The core problem addressed is enabling intuitive, contactless digital drawing using only hand gestures, making creative expression accessible without traditional hardware dependencies



This matters because:

- Democratizes digital art for diverse users
- Enables novel interaction paradigms
- Supports accessibility for motor-impaired individuals
- Prepares for future gesture-based computing interfaces



2. Approach and Technical Solution

- .Core Architecture
- .The solution combines MediaPipe Hands for real-time hand tracking with OpenCV for canvas manipulation, implementing a gesture-driven drawing system:
- .text
- .Input: Webcam → MediaPipe Hand Detection → Gesture Recognition → Tool State Machine → Canvas Operations
- .Gesture Vocabulary
 - Two primary gestures control the entire interface:
 - 1.Tool Selection: Index finger tip (landmark 8) hovers over toolbar → 0.8s dwell time
 - 2.Draw Mode: Index finger PIP (12) raised >40px above middle finger tip (9)

Tool State Machine

text

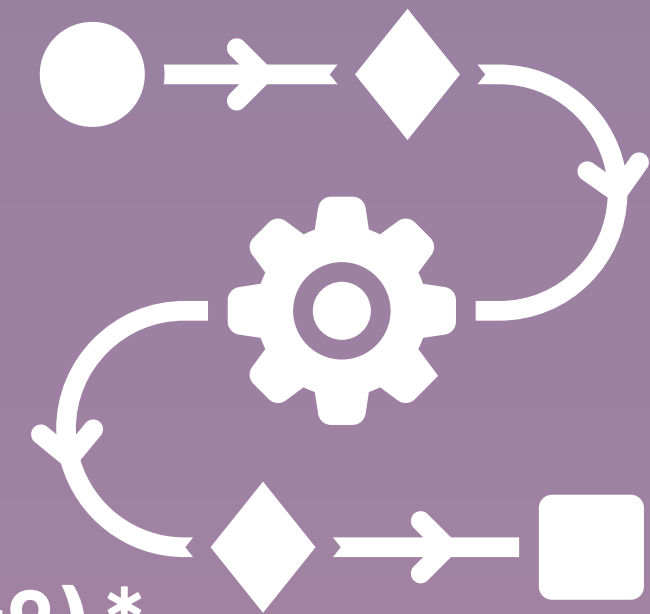
IDLE → TOOL_SELECT (hover+time) → TOOL_ACTIVE → DRAW (finger up) → PREVIEW → CONFIRM (finger down)

Five tools implemented with distinct behaviors

- Line/Rectangle/Circle: Preview on frame → Commit to mask on release
- Draw: Continuous stroke while finger raised
- Erase: 30px circular mask restoration
-

Canvas Management

- Dual-layer rendering prevents destructive drawing:
- python
- `mask = np.ones((frame_h, frame_w), dtype=np.uint8) *`
255 # White = visible
- # Black pixels (0) = erased/hidden content
- `final_frame = bitwise_and(original, original,`
`mask=mask)`



3. Key Technical Decisions

1. Single-Hand Detection Only

Text

```
hands.Hands(..., max_num_hands=1)
```

Rationale: Simplifies gesture recognition, reduces computational load, matches primary use case of single-handed drawing.

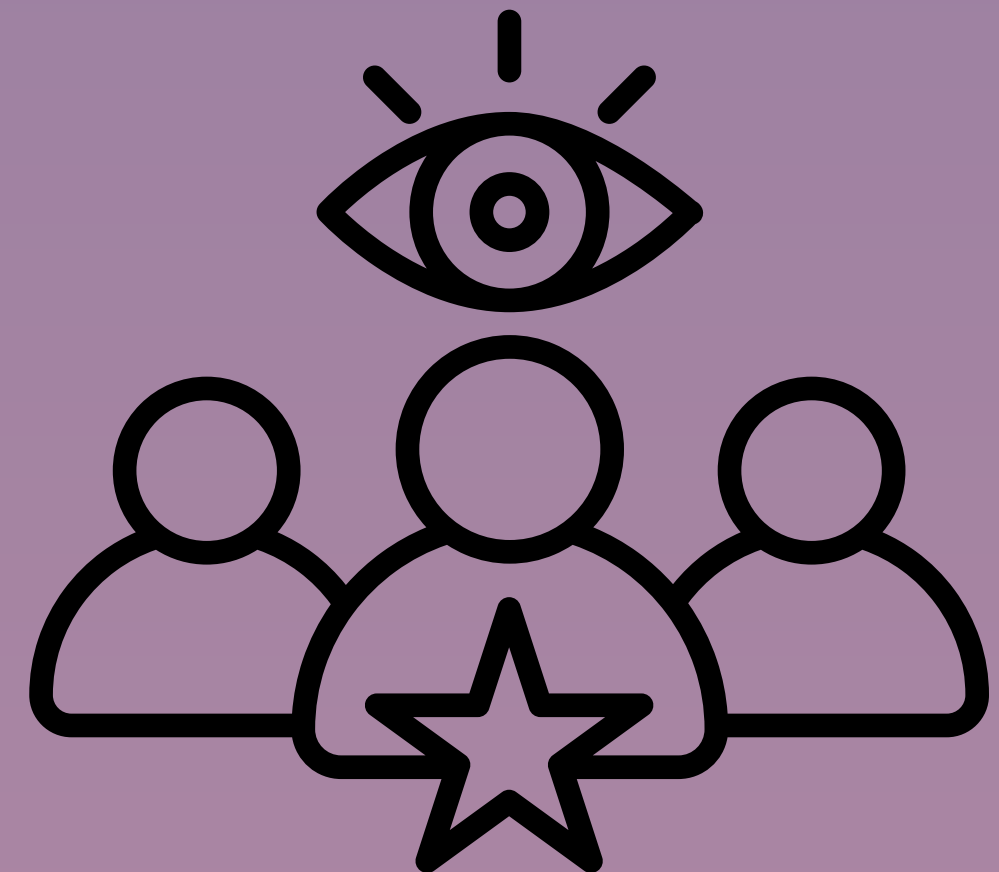
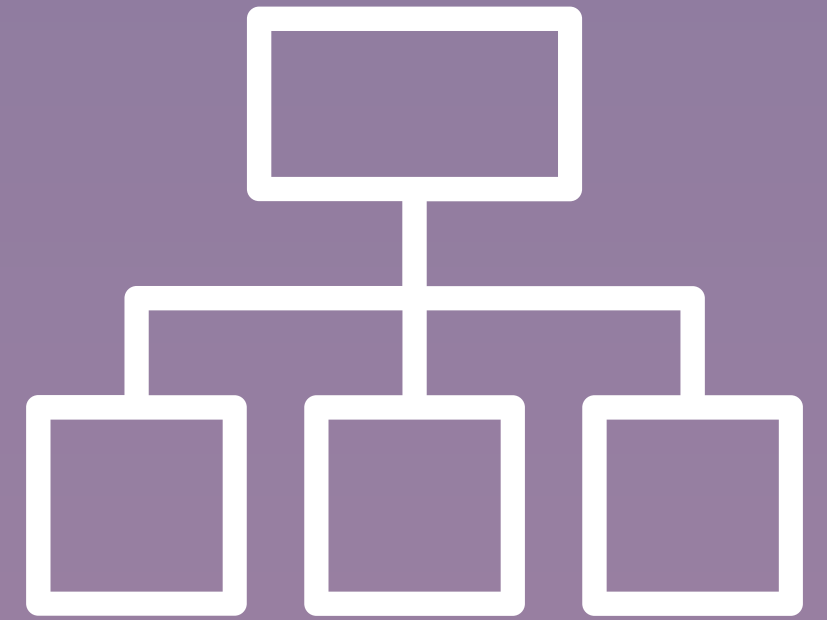
2. Threshold-Based Gesture Recognition

text

```
def index_raised(yi, y9):
```

```
    return (y9 - yi) > 40 # 40px threshold
```

Rationale: Simple, robust vertical distance avoids complex angle calculations, works across hand sizes.



3. Preview-Commit Pattern

text

```
# Preview on original frame (temporary)
```

```
cv2.line(frm, start, end, color, thick)
```

```
# Commit to persistent mask (permanent)
```

```
cv2.line(mask, start, end, 0, thick) # 0=black=hide
```

Rationale: Provides visual feedback during drawing + undo capability via mask restoration.

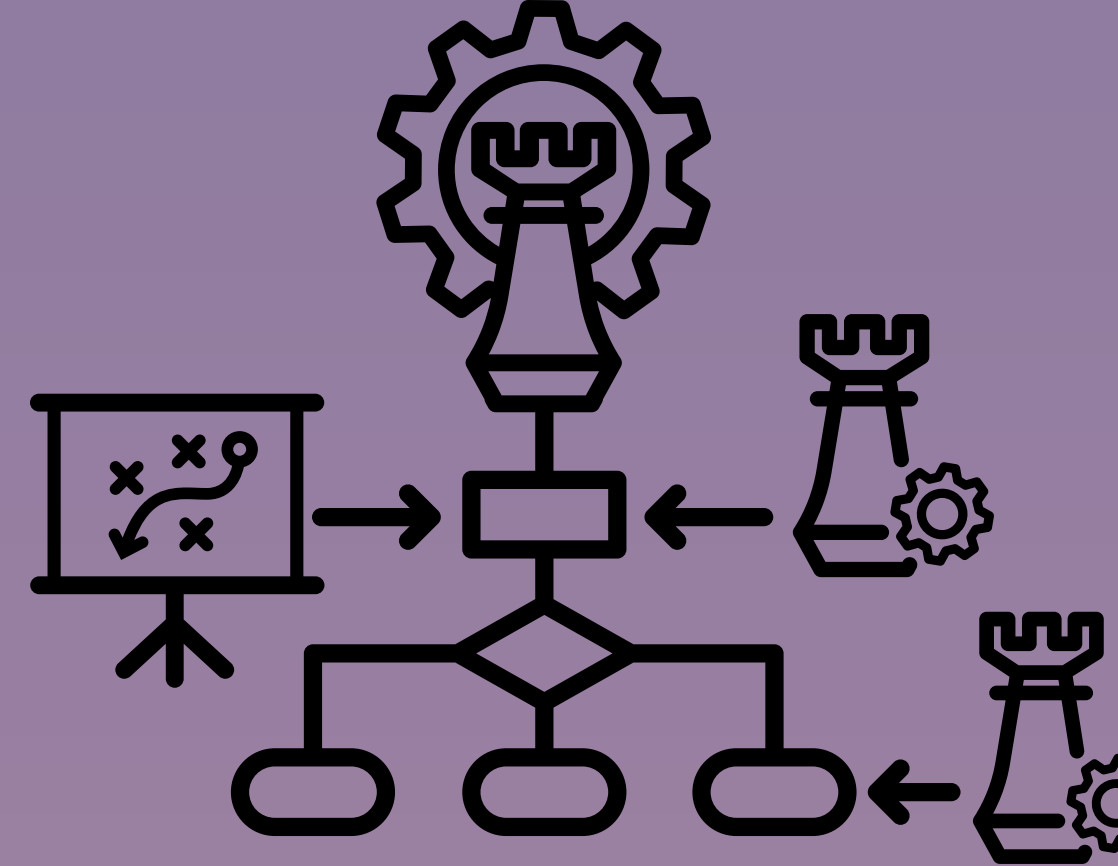
4. Dwell-Time Tool Selection

text

```
if (ptime - ctime) > 0.8: # 800ms hold
```

```
    curr_tool = getTool(x)
```

Rationale: Prevents accidental tool switches while hovering, visual shrinking circle provides feedback.



4. Implementation Challenges and Solutions

Challenge 1: Hand Tracking Stability

Problem: Jittery landmark detection caused erratic drawing

Solution:

High confidence thresholds: `min_detection_confidence=0.6`

Single-hand mode reduces ambiguity

Smooth coordinate tracking via previous frame positions

Challenge 2: Gesture False Positives

Problem: Middle finger movements triggered drawing accidentally

Solution: Strict vertical threshold $(y_9 - y_i) > 40$ instead of angle-based detection

Challenge 3: Tool Boundary Precision

Problem: Toolbar selection failed near edges due to finger width

Solution: 50px tool zones + $x > ml$ margin exclusion: if $x < max_x$ and $y < max_y$ and $x > ml$

Challenge 4: Performance on Commodity Hardware

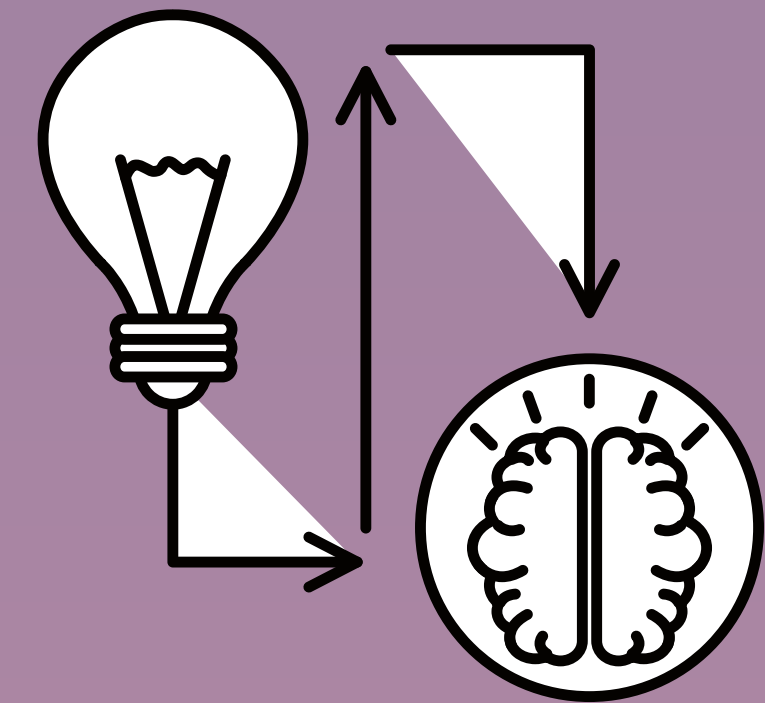
Problem: 30fps target with real-time processing

Solution:

Single-hand detection only

Efficient mask operations

ROI-based toolbar overlay



5. Critical Learnings

Gesture Design Principles

1. Simplicity scales: Two gestures control complex functionality
2. Visual feedback is essential: Shrinking circle, live previews, landmark drawing
3. Thresholds > machine learning: Simple distance metrics outperformed complex models for this use case
4. State machines beat conditionals: Clear TOOL_ACTIVE → DRAW → CONFIRM flow prevents race conditions

Computer Vision Insights

text

Landmark 8 (index tip) = primary interaction point

Landmark 12+9 delta = reliable gesture trigger

Mask operations = perfect for non-destructive edits

UX Patterns for Gestural Interfaces

- Dwell time > rapid gestures for precision selection
- Preview before commit builds user confidence
- Proportional feedback (shrinking circle) shows progress
- Spatial consistency (toolbar stays fixed) aids muscle memory

6. Technical Results



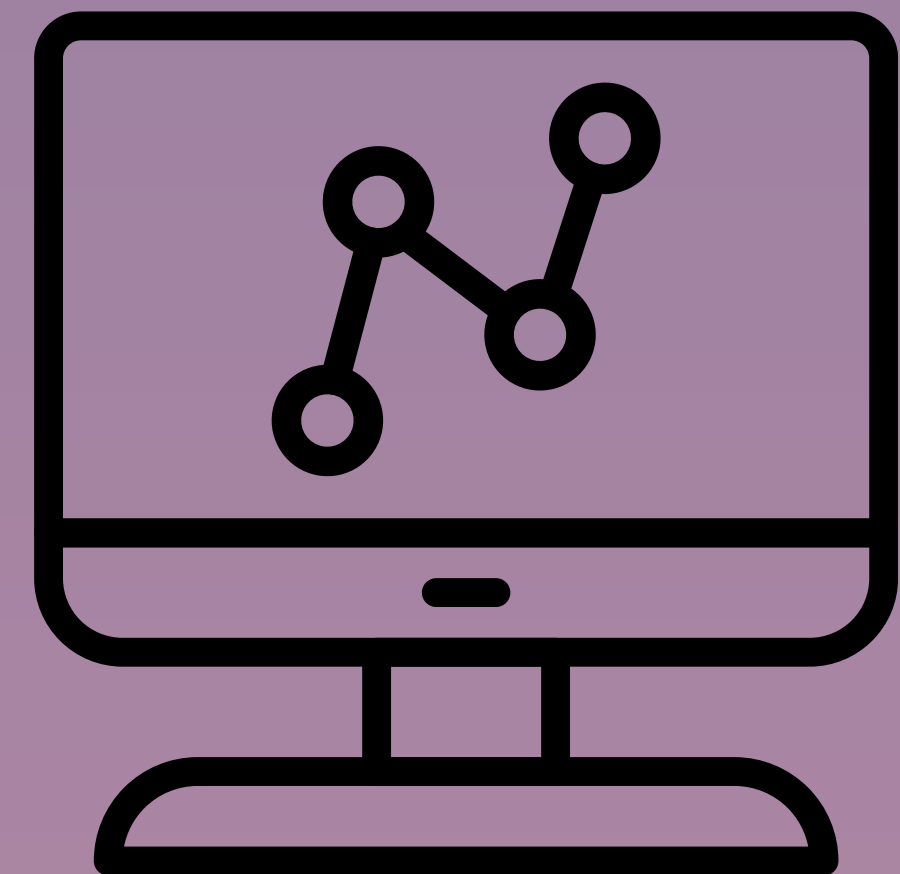
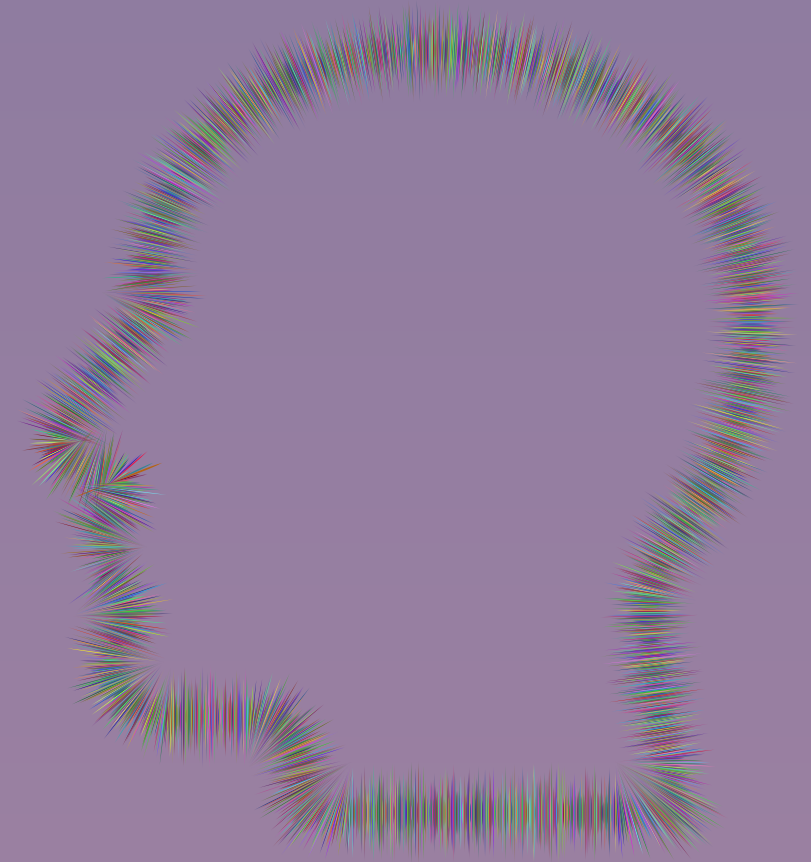
7. Limitations and Future Work

Current Limitations

- Single hand only
- Fixed gesture thresholds (no user calibration)
- Lighting dependent
- No color selection
- No undo/redo stack

Planned Enhancements

1. Multi-hand support: Second hand for color/tools
2. Adaptive thresholds: Calibrate per user hand size
3. ML gesture classifier: Replace hardcoded distances
4. Save/Load canvas: Persistent artwork storage
5. Multi-color palette: Gesture-based color selection

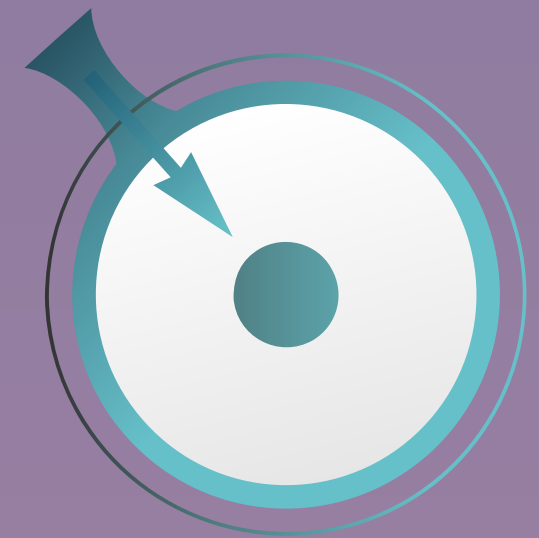


Conclusion

This project successfully demonstrates natural gesture interaction replacing traditional input devices for creative applications. The combination of MediaPipe's robust hand tracking with carefully designed gesture vocabulary creates an intuitive drawing experience accessible to anyone with a webcam.

Key success factors:

- Simple, learnable gesture set (2 gestures, 5 tools)
- Immediate visual feedback throughout interaction
- Non-destructive editing via mask system
- Robust to tracking noise through state management



Thank You

